



ALMA MATER STUDIORUM
UNIVERSITÀ DI BOLOGNA

DIPARTIMENTO DI
INFORMATICA - SCIENZA E INGEGNERIA

INTRODUZIONE ALLA PROGRAMMAZIONE

COSIMO LANEVE

`cosimo.laneve@unibo.it`

CORSO 00819 – PROGRAMMAZIONE

ARGOMENTI (CAPITOLO 1 SAVITCH)

1. sistemi di calcolo
2. programmazione e problem solving
3. introduzione a C++
4. testing e debugging

SISTEMA DI CALCOLO (COMPUTING SYSTEM)

sistema di calcolo = hardware + software

hardware

sono i calcolatori (pc, tablet, laptop, mainframe, etc.)

software

è la collezione di **programmi** che i calcolatori eseguono

programma = sequenza di istruzioni che deve eseguire un calcolatore

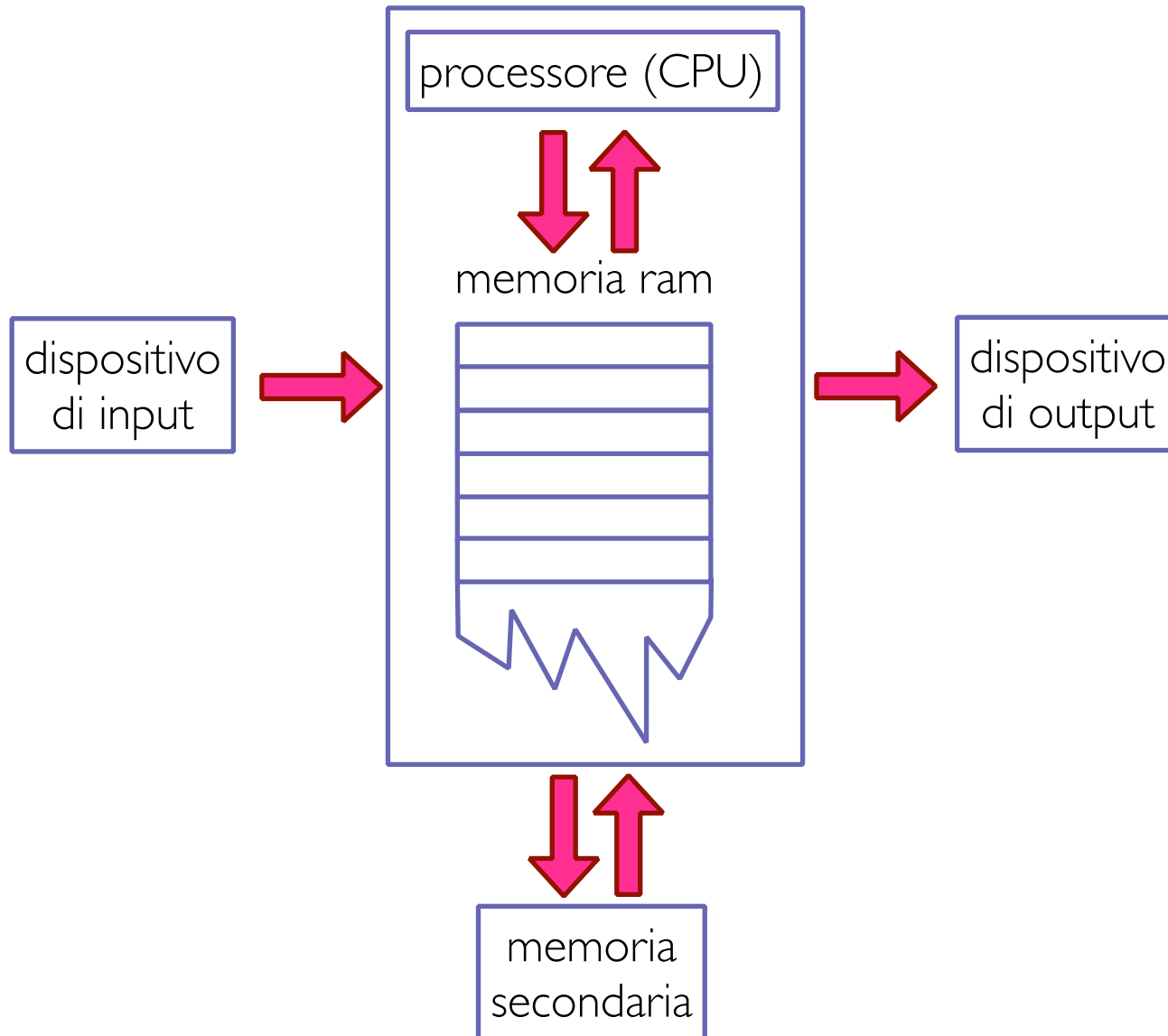
esempi: editors, compilatori, browsers, sistemi operativi

ORGANIZZAZIONE DI UN CALCOLATORE

5 componenti principali

1. **dispositivi di input** (consentono la comunicazione al computer: tastiera, cd, etc.)
2. **dispositivi di output** (consentono la comunicazione dal computer: video, stampante, etc.)
3. **processore** o **CPU** (esegue le istruzioni)
4. **memoria principale** o **RAM** (le locazioni di memoria contengono i programmi in esecuzione)
5. **memoria secondaria** (memorizza i dati in maniera permanente: disco rigido, scheda ssd)

ORGANIZZAZIONE DI UN CALCOLATORE



MEMORIA PRINCIPALE

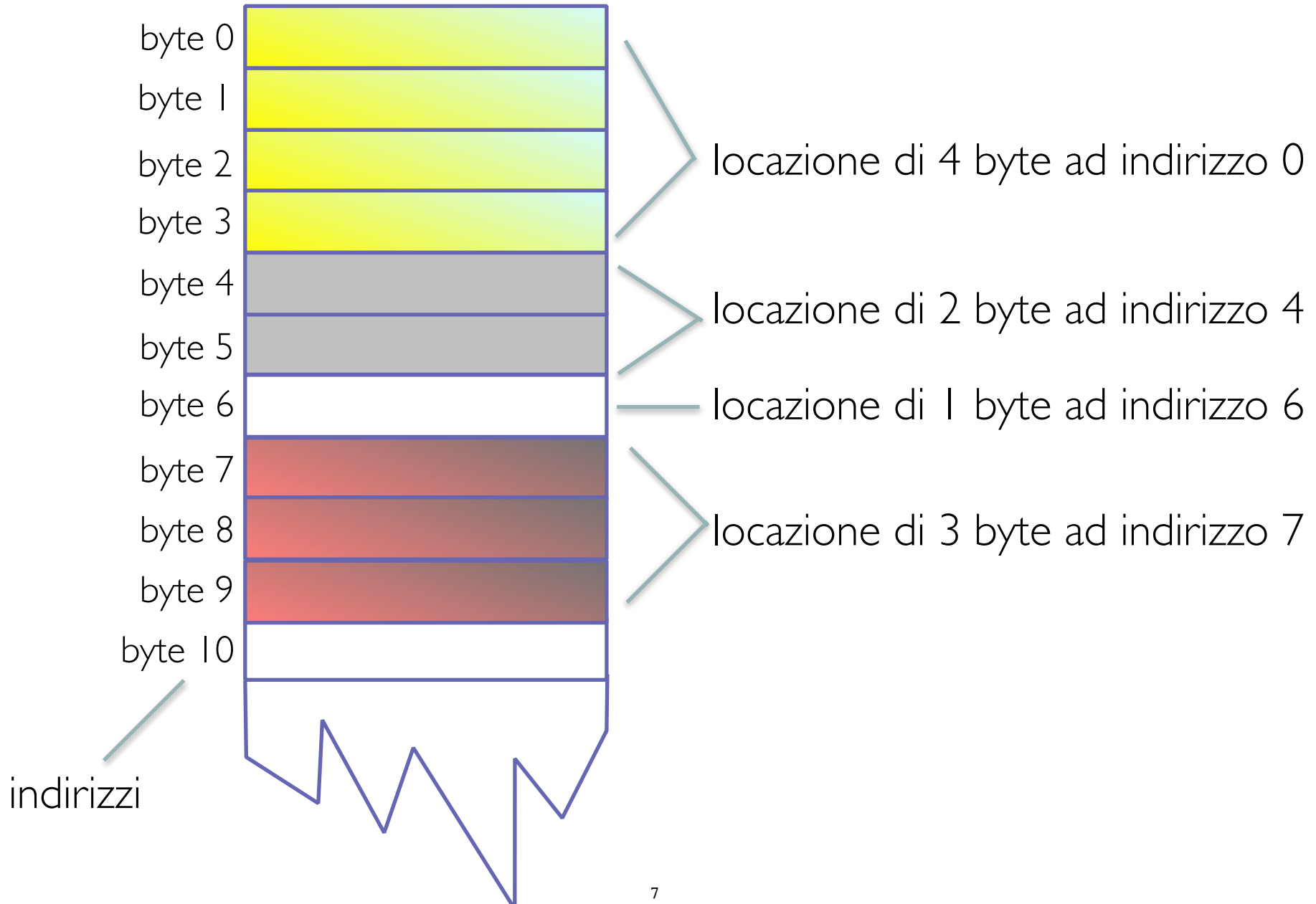
è una lunga **sequenza di locazioni**

- * ognuna contiene più byte (byte = 8 bits, ogni bit è 0 oppure 1)
- * indirizzo: numero che identifica la locazione di memoria

dati grandi: alcuni dati sono troppo grandi per stare in un byte

- * molti interi e reali
- * **gli indirizzi fanno riferimento al primo byte**
- * i byte seguenti servono a memorizzare la parte restante del dato

MEMORIA PRINCIPALE



DATO O CODICE?

- * 'A' può essere memorizzato come 01000001
- * 65 può essere memorizzato come 01000001
- * una istruzione può essere memorizzata come 01000001

come fa il computer a sapere cosa significa 01000001?

- * l'interpretazione dipende dalla istruzione che si sta eseguendo

i programmatori non debbono affrontare queste problematiche (sono troppo di "**basso livello**")

- * i programmatori ragionano come se le celle di memoria contenessero lettere o numeri, piuttosto che zeri e uni

MEMORIA PRINCIPALE E SECONDARIA

la memoria principale memorizza il **programma in esecuzione** e i relativi dati

la **memoria secondaria**

- * memorizza i programmi non in esecuzione, "vecchi"
- * memorizza dati in maniera permanente (è necessaria una esplicita cancellazione per eliminarli)

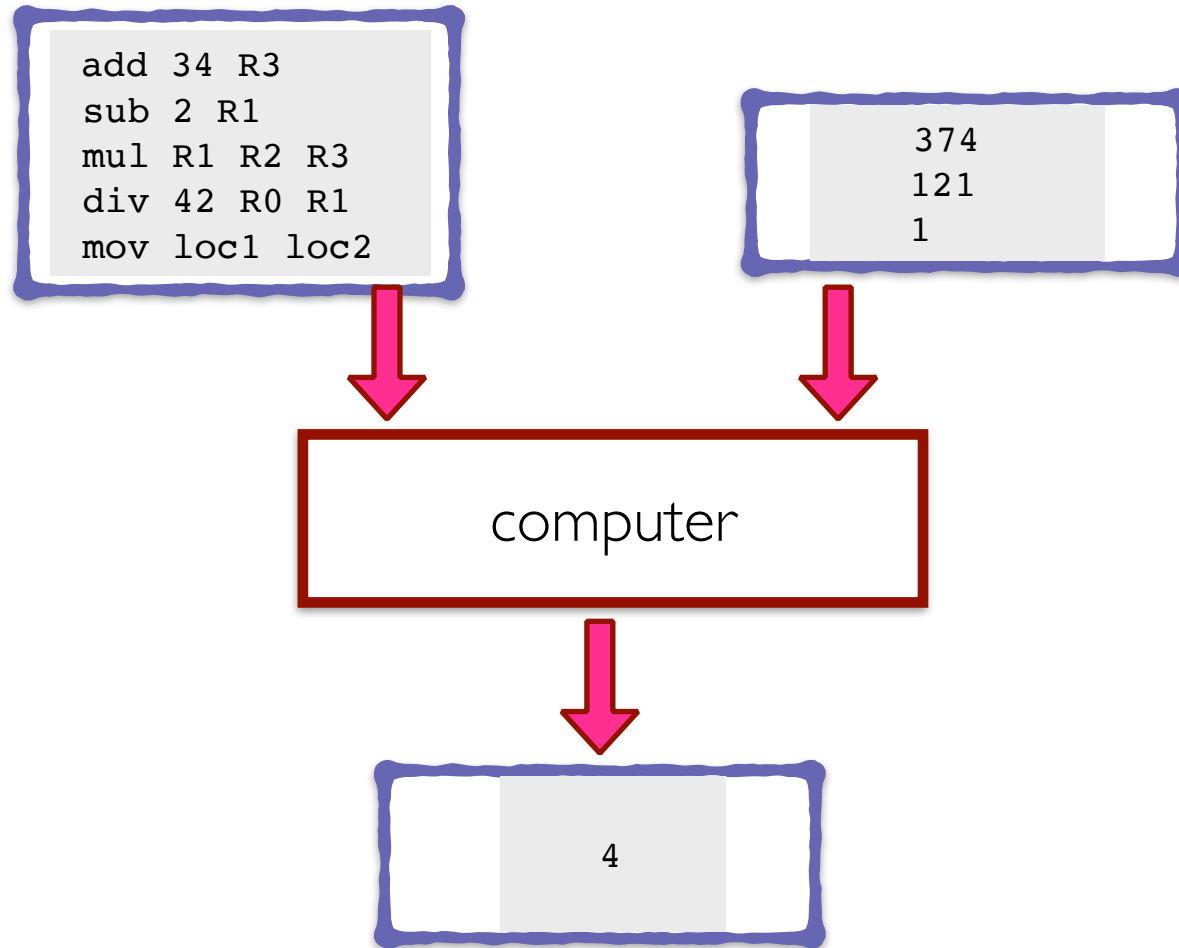
IL PROCESSORE

- * chiamato anche **CPU** — Central Processing Unit
- * esegue le istruzioni del programma: tipiche istruzioni sono (sono esempi, non sono tratti da alcun assembler!)

```
addi 34 28 R3      // R3 = 34+28
add 34.4 28.2 R3    // R3 = 34.4+28.2
addR R1 R2 R3       // R3 = R1+R2
subi 2 5 R1         // R1 = 2-5
mulR R1 R2 R3       // R3 = R1*R2
divi 42 5 R1        // R1 = 42/5 => R1 = 8
mov loc1 loc2       // loc1 e loc2 sono locazioni!
```

ESECUZIONE DI UN PROGRAMMA

(versione semplificata)



LINGUAGGI DI PROGRAMMAZIONE DI BASSO LIVELLO

sono linguaggi per programmare in maniera “**facilmente comprensibile**” per il **calcolatore** (**linguaggi assembler**)

ADD X Y Z

significa: somma il valore nella locazione X al valore nella locazione Y e memorizza il risultato nella locazione Z

- * il codice dei linguaggi assembler deve essere tradotto in **codice macchina** — sequenze di 0 e 1

esempio: 0110 1001 1010 1011

- * le CPU eseguono SOLAMENTE **codice macchina**

LINGUAGGI DI PROGRAMMAZIONE DI ALTO LIVELLO

sono i linguaggi mediante i quali si scrivono programmi

- * assomigliano ai linguaggi naturali (utilizzano costrutti facilmente comprensibili)
- * consentono di programmare in maniera semplice
- * usano istruzioni che sono troppo complicate per le CPU
- * i programmi **devono essere tradotti in codice macchina** che la CPU può eseguire
- * tipici linguaggi sono

C

C++

Java

Visual Basic

PHP

C#

Python

Objective C

I COMPILATORI E I LINKER

i **compilatori** traducono programmi in linguaggi ad alto livello in programmi in codice macchina

source code: è il codice del linguaggio ad alto livello

object code: è il codice in linguaggio macchina

alcuni programmi che usiamo **sono già tradotti in codice oggetto**

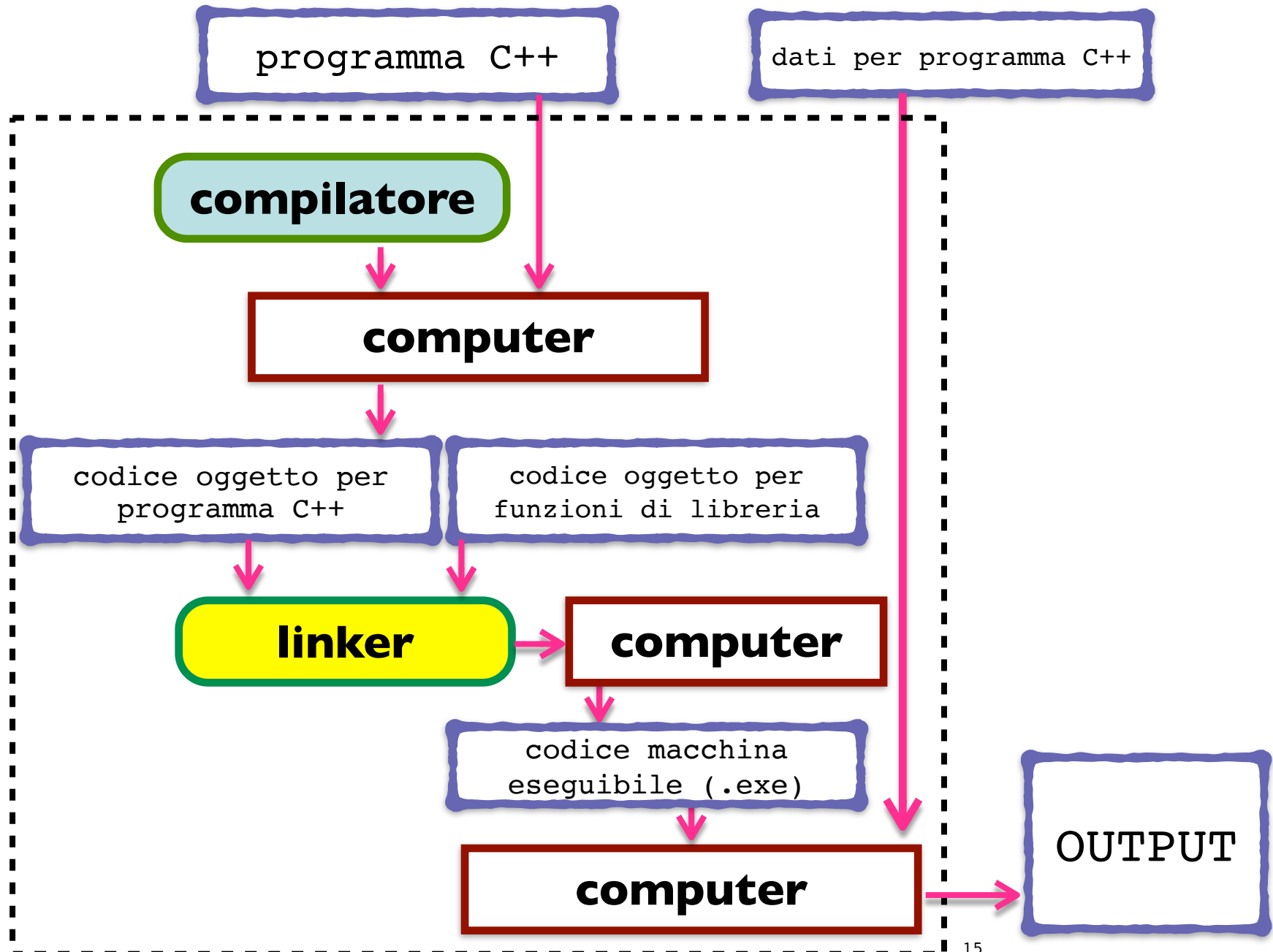
- * funzioni di libreria come le funzioni di input e output

il **linker** combina

- * il codice oggetto del programma che noi scriviamo

- * il codice oggetto pre-compilato delle funzioni di libreria in codice macchina che la CPU può eseguire

COMPILAZIONE ED ESECUZIONE DI UN PROGRAMMA



ALGORITMI E PROGRAMMI

un **algoritmo** è una sequenza di istruzioni che risolve un dato problema

un **programma** è un algoritmo espresso in un **linguaggio di programmazione**

la **programmazione** è un **PROCESSO CREATIVO** che consta di **due fasi**

- * ***problem solving***: si individua un algoritmo che risolve un dato problema
- * ***implementazione***: si traduce l'algoritmo in un programma

OSSERVAZIONE: non ci sono regole per "creare", ma solo indicazioni

Knuth: The **art** of computer programming

FASE DI PROBLEM SOLVING

occorre che il processo sottostante al problema sia completamente specificato

- * quale è l'input?
- * cosa contiene l'output?
- * come sono organizzati l'input e l'output?

definire un algoritmo prima dell' implementazione

- * l'esperienza dice che ciò ci fa guadagnare tempo
- * è più semplice da scrivere
- * è più facile verificarne la correttezza

FASE DI IMPLEMENTAZIONE

1. **tradurre un algoritmo** in un linguaggio di programmazione
 - * è più semplice quando si diventa esperti del linguaggio
2. **compilare** il codice sorgente
 - * controllare e rimuovere gli errori segnalati dal compilatore
3. **esegui il codice** su alcuni casi di test
 - * verifica la correttezza dei risultati
4. i risultati possono **rivelare errori** nell'algoritmo o nel programma (**bugs**) e quindi causare modifiche

TESTING E DEBUGGING

un **bug** è un errore nel programma

- * è un termine introdotto quando una falena causò un guasto in un relay del calcolatore Mark I1 (1947)
- * **debugging** è l'attività di eliminazione degli errori

tipi di errori

- * **errori di sintassi**
 - violazione delle regole grammaticali del linguaggio
 - sono scoperti dal compilatore, la cui messaggistica è spesso imprecisa
- * **errori a run-time**: si verificano durante l'esecuzione (out-of-memory)
- * **errori logici** (sono i più difficili da scoprire)
 - errori nell'algoritmo

DEFINIZIONE DI ALGORITMO

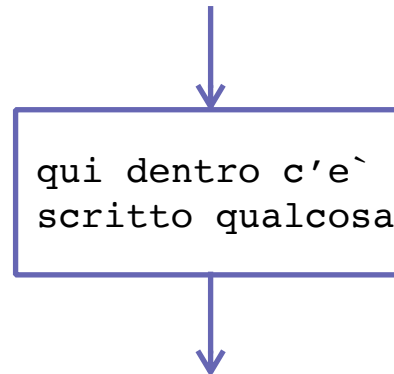
tanti modi

un modo sono i **diagrammi di flusso**
(**flow diagrams**)

DIAGRAMMI DI FLUSSO

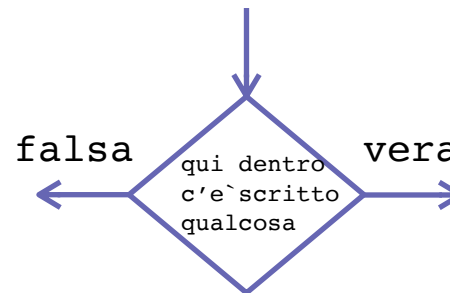
due tipi di nodi:

nodo comando:



- * modifica della memoria
- * input
- * output

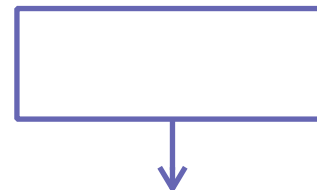
nodo condizione:



- * condizione (ad es. $x > 3$)

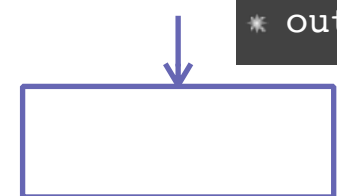
nodi comando speciali:

* input



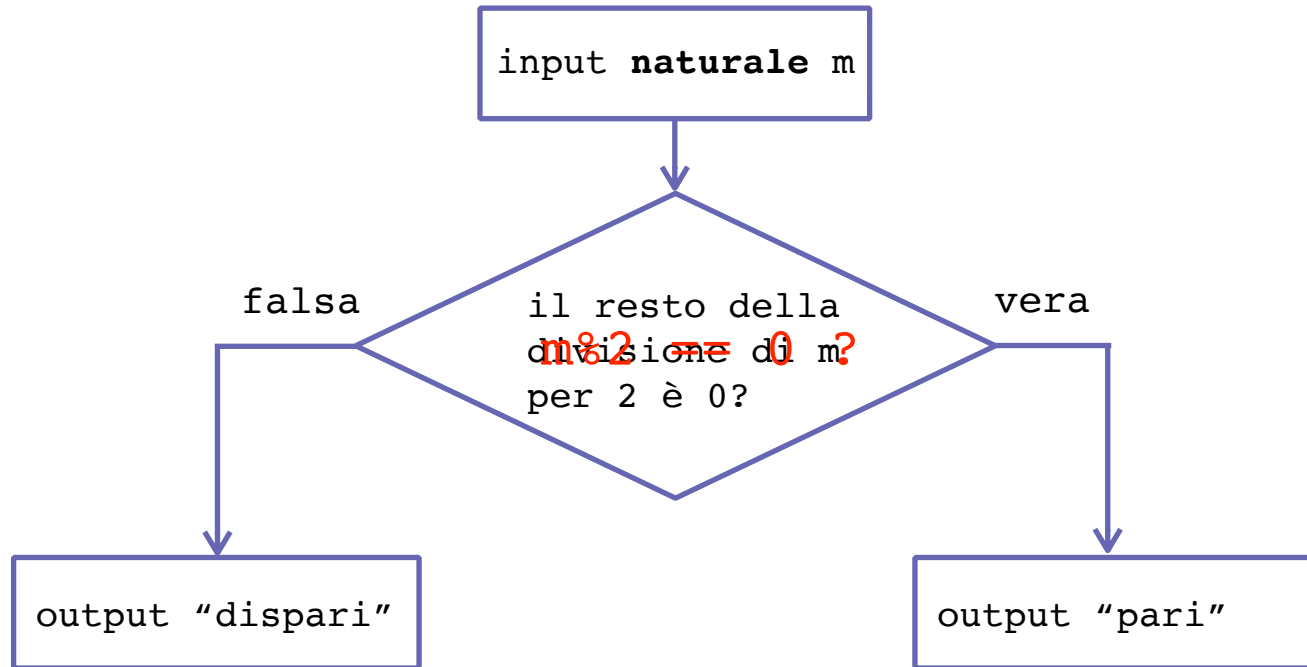
nodo inizio

* output



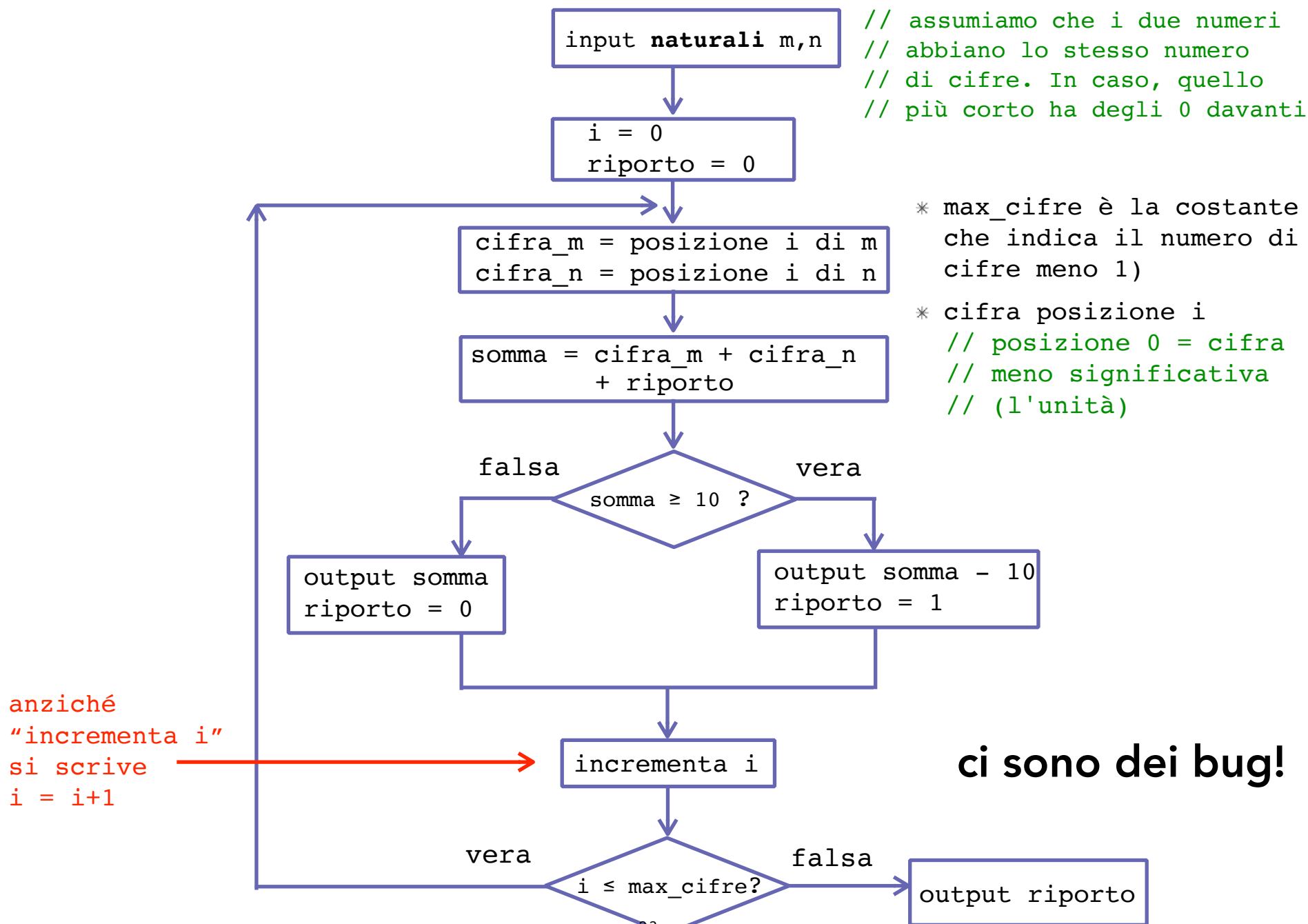
nodo fine

L'ALGORITMO PER PARI O DISPARI

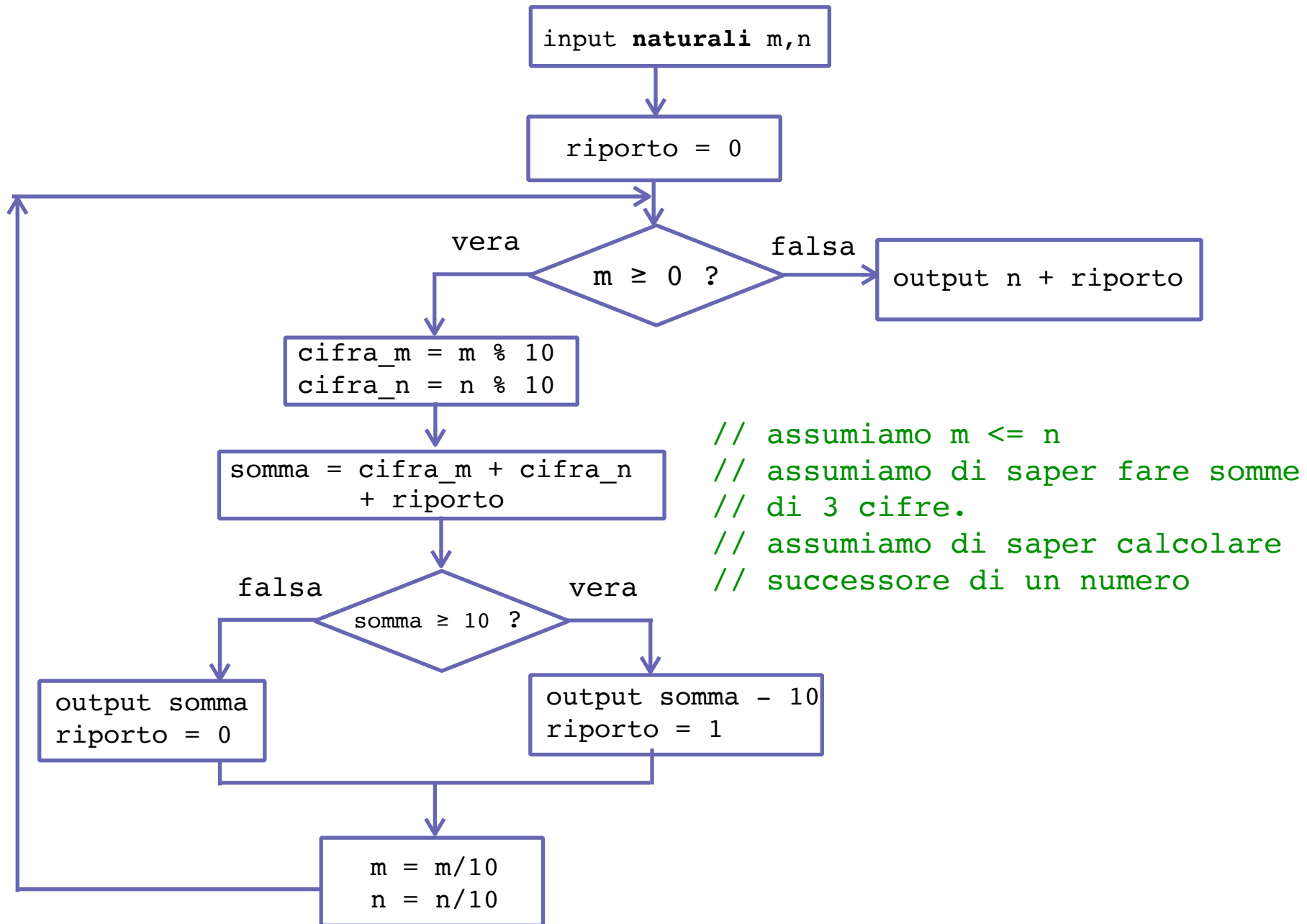


c'è un bug!

L'ALGORITMO DELLA SOMMA DEI NUMERI NATURALI



L'ALGORITMO DELLA SOMMA DEI NUMERI NATURALI



OSSERVAZIONI

- * ci sono **tanti** nomi
- * il valore di questi nomi **cambia durante l'esecuzione**
(**non sono come le variabili matematiche!**)
- * i diagrammi di flusso hanno dei **cicli** (**il calcolo può non terminare?**)

ESERCIZI

scrivere l'algoritmo come diagramma di flusso per questi problemi

1. prendere 3 numeri in input e stampare il loro valore medio
2. prendere 3 numeri in input e stampare il massimo
3. prendere due numeri e stampare il loro prodotto (con l'algoritmo delle scuole elementari, assumendo di saper moltiplicare un numero per una cifra)
4. prendere 4 numeri interi e stampare il valore di essi più vicino alla loro media

PROGRAMMAZIONE OBJECT-ORIENTED

- * abbreviata in OOP
- * è usata in molti linguaggi di programmazione moderni
- * un programma è visto come un insieme di oggetti che contengono valori e le funzioni che possono modificare tali valori
- * progettare un programma significa
 - definire gli oggetti e gli algoritmi delle funzioni corrispondenti

CARATTERISTICHE DI OOP

encapsulation

- * ogni oggetto contiene i propri dati e le funzioni che possono accedere/modificare i dati
- * nessun'altra funzione può accedere (information hiding)

inheritance (ereditarietà)

- * il codice può essere riutilizzato
- * un oggetto può essere definito utilizzando codice di altri oggetti (con modifiche/con aggiunte di nuove funzioni)

polimorfismo

- * un nome di funzione può avere diversi significati a seconda del contesto dove è definito

INTRODUZIONE A C++

da dove salta fuori C++ ?

- * è una estensione di C **con gli oggetti** sviluppata da Bjarne Stroustrup ai Bell Labs nel 1983
- * C, definito da Dennis Ritchie ai Bell Labs nel 1972, è (stato) usato **per mantenere sistemi UNIX** e **per scrivere molte applicazioni commerciali**
- * C deriva dal linguaggio B ([Ken Thompson](#), Bell Labs, [1969](#))
- * B deriva dal linguaggio BCPL (**B**asic **C**ombined **P**rogramming **L**anguage, [Martin Richards](#) dell'[Università di Cambridge](#), [1966](#))

perchè il "++" ?

- * è una abbreviazione molto utilizzata in C++

IL PRIMO PROGRAMMA: HELLO_WORLD

```
int main(){                                     // main() è dove inizia un
                                                // programma C++

cout << "Hello, world!\n";                     // see_out: stampa sul video i
                                                // 13 caratteri Hello, world!
                                                // seguiti da un "vai a capo"

return(0) ;                                    // ritorna un valore che indica
                                                // successo
}
```

osservazioni:

- * i doppi apici delimitano una stringa; `\n` è la notazione per il "vai a capo"
- * notare i `;"` che sono usati per terminare i comandi
- * le parentesi graffe servono per raggruppare comandi in blocchi
- * `main` è una funzione che non prende argomenti `" ( )"` e ritorna un intero (per denotare successo o fallimento)

IL PRIMO PROGRAMMA: HELLO_WORLD COMPLETO

```
#include <iostream>      // usa le librerie necessarie per input/output

using namespace std;     // nell'ultima versione dello standard C++,
                          // i nomi (come cin e cout) sono
                          // partizionati in namespaces. Questa
                          // direttiva using dice che i nomi che usa
                          // il programma sono definiti nel namespace
                          // std (in questo caso l'iostream definisce
                          // il significato di cout e cin in std)

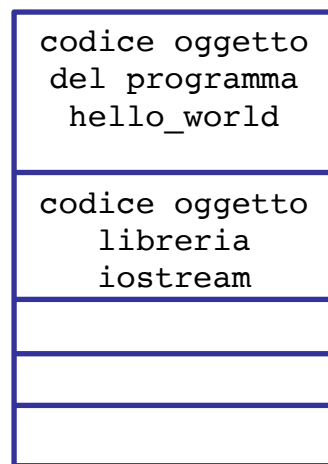
int main(){
    cout << "Hello, world!\n" ;
    return(0) ;
}
```

ESECUZIONE DI HELLO_WORLD

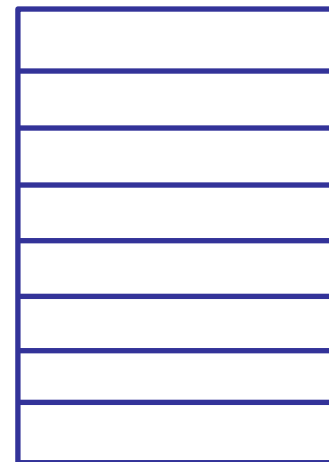
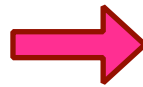
tre passi:

1. il compilatore traduce le istruzioni eseguibili dal C++ al linguaggio macchina
2. il linker include la libreria `iostream`
3. il computer esegue la versione in linguaggio macchina delle istruzioni

esempio di esecuzione



memoria **prima**
dell'esecuzione



memoria **dopo**
dell'esecuzione

output

> Hello, world!

IMPORTANZA DI HELLO_WORLD

`hello_world` è un programma molto importante:

- * il suo obiettivo è di aiutare a familiarizzare con i tool
 - compilatore
 - ambiente di sviluppo (~~Eclipse~~ Clion)
 - ambiente di esecuzione
- * (quando installerete ~~Eclipse~~ Clion) scrivete il programma accuratamente
 - dopo aver visto cosa accade, provate a fare qualche errore e controllate le risposte di ~~Eclipse~~, ad esempio
 - dimenticate il `main()` Clion
 - dimenticate di terminare la stringa `Hello, world!` con i doppi apici
 - scrivete in maniera errata `return`
 - dimenticate una parentesi graffa
 - dimenticate un `“;”`, etc.

ESERCIZI

1. scaricare le slide e studiarle
2. studiare il capitolo 1 del Savitch e fare gli esercizi alla fine del capitolo
3. provare ad installare ~~Eclipse~~ Clion